

文章笔记

构建 Claude Code 的经验：Skills 的设计与使用

基于公开课程资料整理

2026 年 4 月 1 日

原文标题：Lessons from Building Claude Code: How We Use Skills

原文作者：Thariq Shihpar (Anthropic Claude Code 团队工程师, X: @trq212)

原文链接：<https://x.com/trq212/status/2033949937936085378>

发布日期：2026 年 3 月 17 日

译者参考：宝玉 @dotey

项目主页：

导读：本文作者 Thariq Shihpar 是 Anthropic 的 Claude Code 团队工程师，也是 Skills 功能的核心推动者之一。Anthropic 内部活跃使用的 Skills 已达数百个，文中的九大分类体系、编写技巧和分发策略均来自真实的内部实践。本笔记基于英文原文整理，原文比中译版详细近一倍，包含更多技术细节和实操建议。

目录

1 Skills 概述	2
1.1 本章小结	2
2 九大 Skills 类别	2
2.1 类别一: Library & API Reference (库与 API 参考)	2
2.2 类别二: Product Verification (产品验证)	3
2.3 类别三: Data Fetching & Analysis (数据获取与分析)	3
2.4 类别四: Business Process & Team Automation (业务流程与团队自动化)	3
2.5 类别五: Code Scaffolding & Templates (代码脚手架与模板)	4
2.6 类别六: Code Quality & Review (代码质量与审查)	4
2.7 类别七: CI/CD & Deployment (CI/CD 与部署)	4
2.8 类别八: Runbooks (运维手册)	5
2.9 类别九: Infrastructure Operations (基础设施运维)	5
2.10 本章小结	5
3 编写 Skills 的最佳实践	6
3.1 不要说显而易见的事 (Don't State the Obvious)	6
3.2 建一个踩坑点章节 (Build a Gotchas Section)	6
3.3 利用文件系统与渐进式披露 (Use the File System & Progressive Disclosure)	6
3.4 不要把 Claude 限制得太死 (Avoid Railroad Claude)	7
3.5 考虑好初始设置 (Think through the Setup)	7
3.6 description 字段是给模型看的 (The Description Field Is For the Model)	7
3.7 本章小结	7
4 Skills 的高级特性	7
4.1 记忆与数据存储 (Memory & Storing Data)	7
4.2 存储脚本与生成代码 (Store Scripts & Generate Code)	8
4.3 按需钩子 (On Demand Hooks)	8
4.4 本章小结	9
5 Skills 的分发与管理	9
5.1 两种分发方式	9
5.2 管理插件市场 (Managing a Marketplace)	9
5.3 组合 Skills (Composing Skills)	9
5.4 衡量 Skills 的效果 (Measuring Skills)	10
5.5 本章小结	10
6 总结与延伸	10
6.1 核心要点回顾	10
6.2 延伸思考	10
6.3 拓展阅读	11

1 Skills 概述

Skills 已经成为 Claude Code 中使用最广泛的**扩展点**（extension point）之一。它们灵活、容易制作，分发起来也很简单。但也正因为太灵活，实践中会面临三个核心问题：

1. 什么类型的 Skills 值得做？
2. 写出好 Skill 的秘诀是什么？
3. 什么时候该把它们分享给别人？

Anthropic 内部已有数百个 Skills 在活跃使用。本文就是他们从中总结出的经验教训。

Skills 不只是 Markdown 文件

一个常见误解是认为 Skills “只不过是 markdown 文件”。实际上，**Skills 最有趣的地方恰恰在于它们不只是文本文件**。Skills 是**文件夹**，可以包含脚本、资源文件（assets）、数据等内容，智能体可以发现、探索和操作这些内容。在 Claude Code 中，Skills 还拥有丰富的配置选项，包括注册动态钩子（hooks）。Anthropic 发现，最有意思的 Skills 往往创造性地利用了这些配置选项和文件夹结构。

1.1 本章小结

Skills 是 Claude Code 的核心扩展机制，其本质是包含指令、脚本和资源的文件夹，而非简单的文本文件。理解这一点是用好 Skills 的前提。

2 九大 Skills 类别

Anthropic 在梳理了内部所有 Skills 之后发现，它们大致聚集为几个反复出现的类别。**最好的 Skills 清晰地落在某一个类别里；让人困惑的 Skills 往往横跨了好几个类别**。这并非一份穷尽性清单，但它是一个很好的思考框架，帮助你判断组织内是否有遗漏的 Skill 类型。

2.1 类别一：Library & API Reference（库与 API 参考）

帮助正确使用某个库、命令行工具或 SDK 的 Skills。既可以针对内部库，也可以针对 Claude Code 偶尔犯错的常用库。这类 Skills 通常包含：

- 一个文件夹的参考代码片段（reference code snippets）
- Claude 在写脚本时需要避免的踩坑点（gotchas）列表

示例：

- `billing-lib` — 内部计费库：边界情况、`footguns`¹等
- `internal-platform-cli` — 内部 CLI 封装的每个子命令及使用示例
- `frontend-design` — 让 Claude 更好地适配你的设计系统

¹Footgun 是指设计上容易让使用者搬起石头砸自己脚的 API 或功能。

2.2 类别二：Product Verification (产品验证)

描述如何测试或验证代码是否正常工作的 Skills，通常搭配 Playwright、tmux 等外部工具完成验证。

验证类 Skills 值得重点投入

验证类 Skills 对于确保 Claude 输出的正确性**极其有用**。作者明确建议：**值得安排一个工程师花一整周时间专门打磨你的验证 Skills。**

可以考虑的技术手段包括：

- 让 Claude 录制输出过程的视频 (record a video of its output)，这样你可以回看它到底测了什么
- 在每一步强制执行程序化的状态断言 (programmatic assertions on state at each step)
- 在 Skill 中附带各类辅助脚本来实现上述功能

示例：

- `signup-flow-driver` — 在无头浏览器中跑完注册 → 邮件验证 → 引导流程 (onboarding)，每一步都可以插入状态断言的钩子
- `checkout-verifier` — 用 Stripe 测试卡驱动结账 UI，验证发票确实落在正确的状态
- `tmux-cli-driver` — 针对需要 TTY 的交互式 CLI 测试

2.3 类别三：Data Fetching & Analysis (数据获取与分析)

连接到你的数据和监控体系的 Skills。这类 Skills 可能包含：

- 带有凭证的数据获取库
- 特定的仪表盘 ID
- 常见工作流或数据获取方式的说明

示例：

- `funnel-query` — “哪些事件 join 在一起能看到注册 → 激活 → 付费”，加上存放规范 `user_id` 的表
- `cohort-compare` — 对比两个用户群的留存或转化率，标记统计显著的差异 (delta)，链接到分群定义
- `grafana` — 数据源 UID、集群名称、问题到仪表盘的对照表

2.4 类别四：Business Process & Team Automation (业务流程与团队自动化)

把重复性工作流自动化为一条命令的 Skills。通常指令本身比较简单，但可能有较复杂的对其他 Skills 或 MCP 的依赖。

日志驱动的一致性

对于这类 Skills，把之前的执行结果保存在日志文件中可以帮助模型保持一致性，并反思之前的执行情况。例如，每次执行后把结果追加到日志，下次运行时模型就能看到历史上下文。

示例：

- `standup-post` — 汇总任务追踪器、GitHub 活动和之前的 Slack 消息，生成格式化的站会汇报（只包含增量变化，delta-only）
- `create-<ticket-system>-ticket` — 强制执行 schema（有效枚举值、必填字段），加上创建后的工作流（ping reviewer，在 Slack 中发链接）
- `weekly-recap` — 已合并 PR + 已关闭工单 + 部署记录 → 格式化周报

2.5 类别五：Code Scaffolding & Templates（代码脚手架与模板）

为代码库中的特定功能生成框架样板代码的 Skills。你可以把这些 Skills 和可组合的脚本结合使用。当脚手架有自然语言需求、无法纯靠代码覆盖时，这类 Skills 特别有用。

示例：

- `new-<framework>-workflow` — 搭建新的 service/workflow/handler，带上你的 annotations
- `new-migration` — 数据库迁移文件模板加常见踩坑点
- `create-app` — 新建内部应用，预配认证（auth）、日志（logging）和部署配置（deploy config）

2.6 类别六：Code Quality & Review（代码质量与审查）

在团队内部执行代码质量标准并辅助代码审查的 Skills。可以包含确定性的脚本或工具以保证最大程度的可靠性（maximum robustness）。你可能想把这些 Skills 作为 hooks 的一部分自动运行，或放在 GitHub Action 中执行。

示例：

- `adversarial-review` — 生成一个全新视角的子智能体（subagent）来挑刺，实施修复，反复迭代直到发现的问题退化为吹毛求疵（nitpicks）
- `code-style` — 强制执行代码风格，特别是 Claude 默认做不好的那些
- `testing-practices` — 关于如何写测试以及测试什么的指导

Subagent 模式

`adversarial-review` 中的 subagent 是指 Claude Code 在执行任务时启动的另一个独立 Claude 实例。这个“全新视角”（fresh-eyes）的实例没有见过这段代码的上下文，从而避免了原实例的思维惯性，能发现被忽视的问题。

2.7 类别七：CI/CD & Deployment（CI/CD 与部署）

帮助拉取、推送和部署代码的 Skills。这类 Skills 可能引用其他 Skills 来收集数据。

示例：

- `babysit-pr` — 监控 PR → 重试不稳定 CI → 解决合并冲突 → 启用 `auto-merge`
- `deploy-<service>` — 构建 → 冒烟测试 → 渐进式流量切换 (`gradual traffic rollout`) 并对比错误率 → 指标恶化时自动回滚
- `cherry-pick-prod` — 隔离 `worktree` → `cherry-pick` → 冲突解决 → 用模板创建 PR

2.8 类别八：Runbooks（运维手册）

接收一个现象（Slack 消息、告警、错误签名），引导走完多工具排查流程，最后输出结构化报告。

示例：

- `<service>-debugging` — 把症状映射到工具和查询模式，覆盖流量最高的服务
- `oncall-runner` — 拉取告警 → 检查常见嫌疑 (`usual suspects`) → 格式化排查结论
- `log-correlator` — 给定一个 request ID，从所有可能碰过这个请求的系统拉取匹配日志

2.9 类别九：Infrastructure Operations（基础设施运维）

执行日常维护和运维操作的 Skills — 其中一些涉及破坏性操作，需要安全护栏 (`guardrails`)。这类 Skills 让工程师在执行关键操作时更容易遵循最佳实践。

破坏性操作需要安全护栏

基础设施运维类 Skills 可能涉及孤立资源清理、依赖变更等破坏性操作。务必在 Skill 中设计确认流程（如浸泡期 + 用户确认）和安全检查。

示例：

- `<resource>-orphans` — 找到孤立 Pod/Volume → 发到 Slack → 浸泡期 (`soak period`) → 用户确认 → 级联清理
- `dependency-management` — 组织的依赖审批 workflow
- `cost-investigation` — “为什么我们的存储/出口带宽费用突然飙升”，附带具体的 bucket 和查询模式

2.10 本章小结

九大类别覆盖了从开发到运维的全生命周期：Library & API Reference、Product Verification、Data Fetching & Analysis、Business Process Automation、Code Scaffolding、Code Quality & Review、CI/CD & Deployment、Runbooks、Infrastructure Operations。好的 Skill 应当清晰地归属于某一个类别；横跨多个类别的 Skill 往往令人困惑。这个分类框架也可以帮助你审视组织内是否有尚未被覆盖的 Skill 类型。

3 编写 Skills 的最佳实践

确定了要做什么类型的 Skill 之后，接下来的问题是**怎么写**。以下是 Anthropic 总结的最佳实践、技巧和窍门。Anthropic 最近还发布了 Skill Creator 工具，帮助在 Claude Code 中更方便地创建 Skills。

3.1 不要说显而易见的事 (Don't State the Obvious)

Claude Code 对代码库已经非常了解，Claude 本身对编程也很在行，有很多默认的观点和偏好。如果你要发布一个主要提供知识的 Skill，应当聚焦于那些能把 Claude 推离其常规思维方式的信息。

frontend-design Skill 案例

这个 Skill 由 Anthropic 的一位工程师开发，通过与客户反复迭代来改进 Claude 的设计品味。它专门帮 Claude 避免那些经典的套路模式 (classic patterns)，比如 Inter 字体和紫色渐变。这是一个非常好的“非显而易见知识” Skill 的范例：它提供的不是 Claude 已经知道的东西，而是纠正 Claude 的默认倾向。

3.2 建一个踩坑点章节 (Build a Gotchas Section)

Gotchas 是 Skill 中信息量最高的部分

任何 Skill 中最有价值的内容就是踩坑点 (Gotchas) 章节。这些章节应当根据 Claude 在使用你的 Skill 时遇到的常见失败点**逐步积累**。理想情况下，你会随着时间推移持续更新 Skill，把新发现的 gotchas 补充进去。

3.3 利用文件系统与渐进式披露 (Use the File System & Progressive Disclosure)

Skill 是一个文件夹，不只是一个 markdown 文件。你应该把**整个文件系统**当作 context engineering (上下文工程) 和 progressive disclosure (渐进式披露) 的工具。告诉 Claude 你的 Skill 里有哪些文件，它会在合适的时候去读取它们。

Context Engineering 与 Progressive Disclosure

Context engineering 是指精心设计和输入给大语言模型的上下文信息，以最大化输出质量。**Progressive disclosure** 借用 UI 设计概念，指不一次性把所有信息塞给模型，而是让它在需要时再去读取，从而节省上下文窗口空间并减少干扰。

渐进式披露的具体形式：

- **最简形式**：指向其他 markdown 文件让 Claude 按需读取。例如，把详细的函数签名和用法示例拆分到 `references/api.md`
- **模板文件**：如果最终输出是一个 markdown 文件，可以在 `assets/` 中放一个模板文件供复制使用
- **资源文件夹**：提供 `references`、`scripts`、`examples` 等文件夹，帮助 Claude 更高效地工作

3.4 不要把 Claude 限制得太死 (Avoid Railroading Claude)

过度约束的指令会降低 Skill 的复用性

Claude 通常会努力严格遵循你的指令，而 Skills 的复用性很强，所以你需要**小心不要在指令中过于具体**。给 Claude 它需要的信息，但留给它适应具体情况的灵活性 (flexibility to adapt to the situation)。

3.5 考虑好初始设置 (Think through the Setup)

有些 Skills 需要用户提供上下文来完成初始设置。例如，一个发站会汇报到 Slack 的 Skill 可能需要知道发到哪个频道。

推荐做法：

- 把设置信息存在 Skill 目录下的 `config.json` 文件里
- 如果配置未设置好，智能体可以主动向用户询问所需信息
- 如果希望向用户展示**结构化的多选题**，可以让 Claude 使用 `AskUserQuestion` 工具

3.6 description 字段是给模型看的 (The Description Field Is For the Model)

description 不是摘要，而是触发条件

当 Claude Code 启动会话时，它会构建所有可用 Skills 及其 description 的清单。Claude 通过扫描这个清单来判断“这个请求有没有对应的 Skill？”这意味着 **description 字段不是功能摘要** (not a summary)，而是**何时该触发这个 Skill 的描述** (a description of when to trigger)。好的 description 读起来更像一个 if-then 条件，而不是功能说明书。

3.7 本章小结

编写 Skills 的核心原则：聚焦非显而易见的知识（避免重复 Claude 已知的内容）、持续积累 gotchas、利用文件夹结构做 progressive disclosure、避免过度约束 (railroading)、设计好初始配置流程 (`config.json` + `AskUserQuestion`)、把 description 当作触发条件而非功能说明。

4 Skills 的高级特性

4.1 记忆与数据存储 (Memory & Storing Data)

Skills 可以通过在内部存储数据来实现某种形式的**记忆** (memory)。存储方式从简单到复杂：

- 只追加写入的文本日志文件 (append-only text log)
- JSON 文件
- SQLite 数据库

例如，`standup-post` Skill 可以保留一份 `standups.log`，记录它写过的每一条站会汇报。下次运行时，Claude 会读取自己的历史记录，就能知道从昨天到今天发生了什么变化。

数据存储位置：避免升级时丢失

存在 Skill 目录下的数据可能会在升级 Skill 时被删除。应把数据存在稳定的文件夹中。目前 Claude Code 提供了 `#{CLAUDE_PLUGIN_DATA}` 作为每个插件的稳定数据存储目录 (stable folder per plugin)。

4.2 存储脚本与生成代码 (Store Scripts & Generate Code)

能给 Claude 的最强大工具之一就是**代码**。给 Claude 提供脚本和库，让它把精力花在**组合编排** (composition) 上 — 决定下一步做什么，而不是从头重建样板代码。

例如，在数据科学 Skill 中放一组从事件源获取数据的辅助函数库：

```
1 # helpers/data_fetcher.py
2 def get_events(start_date, end_date, event_type):
3     """从事件源获取指定时间范围和类型的事件数据"""
4     ...
5
6 def get_user_cohort(cohort_definition):
7     """根据定义获取用户群"""
8     ...
9
10 def compute_retention(cohort, period="7d"):
11     """计算指定用户群的留存率"""
12     ...
```

Listing 1: Skill 中辅助函数库示例：数据获取与分析

Claude 可以即时生成脚本来组合这些函数，完成更高级的分析。例如，面对“周二发生了什么？”这样的提问，Claude 可以自动编排多个辅助函数来回答。

4.3 按需钩子 (On Demand Hooks)

Skills 可以包含只在该 Skill 被调用时才激活的钩子，并在整个会话期间保持生效。这适合那些**比较主观** (opinionated) 的钩子 — 你不想让它们一直运行，但在特定场景下极其有用。

示例：

- `/careful` — 通过 `PreToolUse` 匹配器拦截 Bash 中的 `rm -rf`、`DROP TABLE`、`force-push`、`kubectl delete` 等危险操作。你只在碰生产环境时才需要它 — 如果一直开着会让人抓狂 (“having it always on would drive you insane”)
- `/freeze` — 阻止对特定目录之外的任何 Edit/Write 操作。调试时很有用：“我想加日志但又怕不小心‘修’了不相关的代码”

PreToolUse 钩子机制

`PreToolUse` 是 Claude Code 的钩子之一，在 Claude 每次调用工具之前触发。可以在这个钩子里检查即将执行的命令，如果命中危险操作就阻止执行。按需钩子 (On Demand Hooks) 让这种检查只在用户主动调用特定 Skill 时才激活，避免在日常开发中造成干扰。这是 Skills 配置选项的一个精妙应用。

4.4 本章小结

高级特性让 Skills 从简单的指令文件升级为功能完整的开发工具：利用日志、JSON 或 SQLite 实现跨会话记忆（注意使用 `CLAUDE_PLUGIN_DATA` 避免数据丢失）；提供辅助脚本让 Claude 专注于组合编排而非重建样板；通过按需钩子在需要时激活安全检查而不干扰日常工作。

5 Skills 的分发与管理

5.1 两种分发方式

Skills 最大的好处之一是可以与团队其他成员共享。有两种方式：

1. **提交到代码仓库**：把 Skills 放在 `./.claude/skills` 下。适合在较少代码仓库上协作的小团队，但每个被 check in 的 Skill 都会给模型上下文增加一点负担。
2. **内部插件市场**：搭建 Claude Code Plugin Marketplace，让用户可以上传和安装插件。随着规模扩大，内部插件市场让团队成员自己决定安装哪些 Skills，避免上下文膨胀。

5.2 管理插件市场 (Managing a Marketplace)

怎么决定哪些 Skills 进入市场？人们怎么提交？

Anthropic 内部**没有一个中心团队**来做决定，而是让最有用的 Skills 有机地 (organically) 涌现出来：

1. 如果你有一个想让别人试试的 Skill，先上传到 GitHub 的 `sandbox` 文件夹，然后在 Slack 等论坛推广
2. 当 Skill 获得了足够的关注（由 Skill 作者自己判断），就可以提交 PR 将其移到正式的 marketplace
3. 在正式发布前确保有审核机制

质量控制不可缺少

创建质量差或重复的 Skills 非常容易 (“it can be quite easy to create bad or redundant skills”)。在正式发布前确保有某种审核机制 (curation method) 非常重要。

5.3 组合 Skills (Composing Skills)

Skills 之间可以互相依赖。例如，你可能有一个文件上传 Skill 负责上传文件，还有一个 CSV 生成 Skill 负责生成 CSV 并调用上传。这种依赖管理目前**还没有被原生支持** (not natively built into marketplaces or skills yet)，但你可以直接按名字引用其他 Skills — 只要对方已安装，模型就会调用它们。

5.4 衡量 Skills 的效果 (Measuring Skills)

用 PreToolUse 钩子追踪 Skill 使用情况

Anthropic 使用一个 PreToolUse 钩子在公司内部记录 Skill 的使用情况。这让他们能够发现哪些 Skills 受欢迎，以及哪些 Skills 的触发频率低于预期 (undertriggering)。这是理解 Skill 生态健康状况的关键手段。

5.5 本章小结

Skills 的分发有仓库级和市场级两种方式，适合不同规模的团队。管理的核心思路是让好的 Skills 自然涌现而非自上而下指定，同时做好质量控制。Skills 之间可以按名字互相引用实现组合，但原生依赖管理尚待完善。通过 PreToolUse 钩子追踪使用数据是衡量 Skill 生态健康的有效手段。

6 总结与延伸

6.1 核心要点回顾

本文是 Anthropic Claude Code 团队基于内部数百个 Skills 的实战经验总结。核心要点包括：

1. **Skills 是文件夹，不是文件**：包含脚本、资源、数据和配置，而非仅仅是一段 Markdown 指令。最有趣的 Skills 创造性地利用了文件夹结构和配置选项。
2. **九大类别**：Library & API Reference、Product Verification、Data Fetching & Analysis、Business Process Automation、Code Scaffolding、Code Quality & Review、CI/CD & Deployment、Runbooks、Infrastructure Operations。好的 Skill 应归属单一类别。
3. **编写原则**：聚焦非显而易见的知识、持续积累 gotchas、利用文件夹做 progressive disclosure、避免 railroading、设计好初始配置 (config.json)、把 description 当触发条件。
4. **高级特性**：通过日志/数据库实现记忆存储（使用 CLAUDE_PLUGIN_DATA）、提供辅助脚本让 Claude 专注于组合编排、通过 On Demand Hooks 在需要时激活安全检查。
5. **分发与管理**：从仓库提交到内部插件市场的演进路径，让好的 Skills 自然涌现，注重质量审核，用 PreToolUse 钩子追踪使用数据。

6.2 延伸思考

正如作者 Thariq Shihpar 所言：

Skills are incredibly powerful, flexible tools for agents, but it's still early and we're all figuring out how to use them best. Think of this more as a grab bag of useful tips that we've seen work than a definitive guide.

Skills 作为 AI 智能体的扩展机制还处于早期阶段。与其把本文当作权威指南，不如视为经过实践验证的技巧合集。大多数好的 Skills 一开始只是几行文字加一个 gotcha，后来因为不断补充 Claude 遇到的新边界情况，才慢慢完善起来。

最好的学习方式就是动手开始 (The best way to understand skills is to get started, experiment, and see what works for you)。

6.3 拓展阅读

- Claude Code 官方文档中的 Skills 章节
- Skilljar 平台上关于 Agent Skills 的课程
- Claude Code Plugin Marketplace 文档
- Skill Creator — Anthropic 发布的 Skills 创建工具
- Thariq Shihpar 的 X 账号 (@trq212) — 持续分享 Claude Code 使用经验
- 原文链接: Lessons from Building Claude Code: How We Use Skills